The `offset` method goes hand in hand with the `limit` method. It allows us to specify the offset of the first row in the result set that will be returned:

```
# The view wants to display orders grouped into pages,
# where each page shows page_size orders at a time.
# This method returns the orders on page page_num (starting
# at zero).
def Order.find_on_page(page_num, page_size)
  order(:id).limit(page_size).offset(page_num*page_size)
end
```

We can use `offset` in conjunction with `limit` to step through the results of a query *n* rows at a time.

select

By default, `ActiveRecord::Relation` fetches all the columns from the underlying database table—it issues a `select * from...` to the database. Override this with the `select` method, which takes a string that will appear in place of the `*` in the `select` statement.

This method allows us to limit the values returned in cases where we need only a subset of the data in a table. For example, our table of podcasts might contain information on the title, speaker, and date and might also contain a large BLOB containing the MP3 of the talk. If you just wanted to create a list of talks, it would be inefficient to also load the sound data for each row. The `select` method lets us choose which columns to load:

```
list = Talk.select("title, speaker, recorded_on")
```

joins

The `joins` method lets us specify a list of additional tables to be joined to the default table. This parameter is inserted into the SQL immediately after the name of the model's table and before any conditions specified by the first